

Activity #1

Linux and Git Cheat Sheet:

1.Linux Command (10)

Command	Sample Usage	What it does
ls	<code>ls -l /home</code>	List files & folders
pwd	<code>pwd</code>	Show current directory path
cd	<code>cd Documents / cd ..</code>	Change directory
mkdir	<code>mkdir my_project</code>	Make a new folder
rm	<code>rm -rf temp_folder</code>	Delete files/folders
cp	<code>cp file.txt ~/Backup/</code>	Copy files
mv	<code>mv oldname.txt newname.txt</code>	Move / rename files
cat	<code>cat README.md</code>	Show file contents
clear	<code>clear</code>	Clear terminal screen
sudo	<code>sudo apt update</code>	Run command as admin

2.Basic Git commands (command, sample usage, description (use your own wordings), common options/flags

Command	Sample Usage	Description (Own Words)	Common Options/Flags
git init	git init my_project	Make ordinary folder into a new, empty Git repository so Git can track the changes inside it.	-q It the hide messages --bare Make a repo without a working file
git status	git status git status -s	Show what happen at files that change and show what file are you on.	-s Short compact format -b Show branch name --ignored Show ignored files
git add	git add index.html git add . git add *.js	Moves your file changes into the staging area — prepares them to be saved in the next commit.	-A Stage ALL changes . Stage current folder only -p Pick changes interactively
git commit	git commit -m "Add homepage" git commit -a -m "Update links"	Saves everything staged permanently into history with a unique ID and a message explaining what changed.	-m "msg" Add commit message -a Stage+commit tracked files together --amend Edit last commit
git diff	git diff git diff --staged	Shows exactly which lines were added, removed, or changed — lets you review edits before saving.	--staged Compare staged vs last commit --color-words Highlight word changes HEAD~1 HEAD Compare last two commits

git branch	git branch git branch feature-new git branch -d old	Lists, creates, renames, or deletes branches — lets you work on different features without messing up main code.	-a List all branches (local+remote) -d Delete safely -D Force delete -m Rename branch
git merge	git merge feature-new git merge --abort	Combines two branches together — takes changes from another branch and merges into your current one.	--no-ff Always create merge commit --squash Combine into one commit --abort Cancel merge safely
git tag	git tag v1.0.0 git tag -a v1.1 -m "Release v1.1"	Marks important commits like version numbers or release points — easy to find specific versions later.	-a Create annotated tag (with message) -d Delete tag -l List all tags
git clone	git clone https://...git git clone <url> my-folder	Downloads an entire existing project from GitHub or any server to your computer — gets all files & history	-b <branch> Clone only one branch --depth 1 Download only latest version <url> <folder> Save to custom folder name

3.the three-tree architecture of Git

Tree / Layer	Purpose	Commands that move changes HERE
Working Directory	Your actual files on computer — where you edit	Files you create/edit manually
Staging Area (Index)	Middle layer — what <i>will</i> go into next commit	git add → moves here
Local Repository	All saved commits, history & branches	git commit → moves staged changes HERE